

Documento de Modelo — Assistente e Módulo Admin

1. Objetivo

Este documento registra, de forma explicativa, o entendimento atual sobre:

- o funcionamento do banco e do fluxo do repositório do assistente;
- a proposta de convivência entre o assistente e o futuro módulo administrativo;
- as decisões de modelagem já discutidas;
- os pontos que ainda podem evoluir no futuro.

O objetivo é ajudar:

- a preservar contexto;
- a apoiar futuras definições de requisitos;
- a orientar a implementação do módulo **admin**;
- a facilitar o entendimento para participantes atuais do projeto e para qualquer pessoa que entre depois e precise de contexto.

Este documento não substitui um documento formal de requisitos. Ele deve ser lido como uma referência técnica e de arquitetura em evolução.

Ele consolida uma modelagem inicial já discutida no projeto, análises anteriores produzidas sobre o módulo administrativo e o entendimento atual do funcionamento do assistente.

2. Contexto Geral

Hoje já existe um projeto funcional de assistente de pesquisa em saúde. Esse assistente:

- possui frontend e backend;
- possui autenticação de usuários;
- possui fluxo de criação de documento de projeto;
- possui fluxo de engenharia/limpeza de dados;
- gera artefatos como **.docx**, **.py** e **.zip**;
- usa banco PostgreSQL no estado atual do repositório original.

A necessidade atual não é reescrever todo o assistente, e sim:

- entender o modelo existente;
- criar a camada administrativa sem quebrar muito o que já funciona;
- preparar uma base mais clara para a evolução da plataforma.

3. Leitura do Assistente Atual

3.1. Ideia central

O assistente atual não foi modelado originalmente com uma entidade formal de **project** como centro do domínio.

Na implementação atual, o eixo principal é a tabela **wizard_sessions**, que representa sessões guiadas do assistente.

Essas sessões guardam:

- o estado do fluxo;
- o título da sessão;
- os dados estruturados do documento;
- o script de limpeza, quando houver;
- o resultado de validação;
- o histórico de mensagens;
- os artefatos exportados.

Na prática, hoje o “projeto” do assistente está fortemente embutido na sessão do tipo **project_doc**.

3.2. Tipos de sessão

O assistente possui atualmente dois fluxos principais, ambos representados por **wizard_sessions**.

project_doc

Representa o fluxo de construção do documento do projeto de pesquisa.

Esse fluxo serve para:

- estruturar contexto;
- definir perguntas de pesquisa e hipóteses;
- registrar objetivos;

- descrever fontes de dados;
- descrever população do estudo;
- registrar variáveis e desfechos;
- descrever métodos e plano de análise;
- descrever aspectos de IA responsável;
- listar artefatos esperados;
- exportar o documento final em **.docx**.

data_clean

Representa o fluxo de engenharia/limpeza de dados.

Esse fluxo serve para:

- escolher o dataset;
- vincular a sessão a um projeto;
- explorar esquema e tabelas;
- discutir coorte, joins e transformações;
- gerar um rascunho de script **data_clean.py**;
- validar esse script;
- exportar o script.

Importante:

- **data_clean** não deve ser entendida como geração automática final do dataset padrão ouro;
- ela funciona hoje mais como uma proposta técnica ou rascunho de preparação de dados;
- os scripts gerados ainda exigem revisão humana antes de uso em produção.

3.3. Tabelas atuais do assistente

Com base no código atual, o assistente possui, entre outras, as seguintes tabelas:

- **users**
- **app_settings**
- **datasets**
- **catalog_tables**
- **catalog_columns**
- **table_relationships**

- **wizard_sessions**
- **chat_messages**
- **export_artifacts**
- **cleaning_versions**
- **audit_logs**

3.4. O que cada tabela representa

users

Guarda os usuários do sistema.

Campos principais atuais:

- **id**
- **email**
- **full_name**
- **hashed_password**
- **role**
- **is_active**
- **created_at**

Hoje o enum de papel é simples:

- **researcher**
- **admin**

app_settings

Guarda configurações gerais da aplicação em formato chave/valor.

datasets

Guarda os datasets cadastrados no sistema.

catalog_tables

Guarda as tabelas do catálogo de um dataset.

catalog_columns

Guarda as colunas das tabelas do catálogo.

table_relationships

Guarda relações entre tabelas do catálogo.

wizard_sessions

É a tabela principal do assistente hoje.

Campos relevantes:

- **id**
- **user_id**
- **wizard_type**
- **current_step**
- **title**
- **dataset_id**
- **linked_project_id**
- **section_data**
- **script_content**
- **validation_result**
- **quality_checklist**
- **llm_model_used**
- **created_at**
- **updated_at**

chat_messages

Guarda mensagens de chat de uma sessão.

export_artifacts

Guarda arquivos gerados pelo sistema.

Exemplos de **artifact_type** observados no código:

- **project_docx**
- **submission_bundle**
- **data_clean_py**

cleaning_versions

Guarda versões do script de limpeza de dados.

audit_logs

Guarda trilha de auditoria do sistema.

3.5. Relações relevantes no assistente atual

users -> wizard_sessions

Um usuário pode ter várias sessões.

datasets -> catalog_tables -> catalog_columns

Representa o catálogo de estruturas de dados.

wizard_sessions -> chat_messages

Uma sessão possui várias mensagens.

wizard_sessions -> export_artifacts

Uma sessão pode gerar vários artefatos.

wizard_sessions -> cleaning_versions

Uma sessão de limpeza pode possuir várias versões do script.

wizard_sessions.linked_project_id

Hoje esse campo aponta para outra **wizard_session**.

Na prática, ele é usado para ligar:

- uma sessão **data_clean**
- à sessão **project_doc** correspondente

Ou seja:

- hoje a ligação entre documento do projeto e engenharia de dados ainda é feita dentro do universo das próprias sessões do assistente.

3.6. Como o documento do projeto funciona hoje

Hoje o documento do projeto pode aparecer de duas formas.

Documento importado pelo usuário

O assistente permite importar um **.docx**.

Nesse caso:

- o arquivo é lido;
- o texto é extraído;
- o conteúdo é distribuído em seções;
- essas seções passam a compor o **section_data** da sessão.

Importante:

- o documento importado não parece ser persistido hoje como anexo formal separado no banco;
- ele é usado principalmente como fonte para preencher a sessão.

Documento exportado pelo sistema

Quando o usuário exporta o projeto:

- o sistema monta um **.docx** com base no **section_data**;
- salva esse arquivo em disco;
- registra esse arquivo em **export_artifacts** como **project_docx**.

Então:

- documento importado e documento exportado são conceitos diferentes no estado atual do assistente.

3.7. Como o bundle de submissão funciona hoje

Hoje o assistente também gera um pacote **.zip** para submissão.

Esse bundle inclui, pelo menos:

- **projeto.docx**
- **data_clean.py**

- um arquivo **LEIA-ME**

Atualmente:

- o sistema gera esse pacote;
- registra o arquivo como **submission_bundle** em **export_artifacts**;
- e devolve o **.zip** para download do usuário.

Isso faz sentido hoje como mecanismo de segurança operacional, mas no futuro tende a perder protagonismo em favor de um fluxo mais integrado de envio à comissão.

4. Entendimento do Fluxo de Negócio

4.1. O módulo admin não cria o projeto

Este é um ponto central.

Na visão atual discutida:

- o pesquisador se cadastra;
- o administrador aprova o acesso;
- o pesquisador entra no assistente;
- o projeto nasce a partir da sessão do assistente;
- o admin não cria o projeto do zero manualmente.

Portanto:

- o **admin** deve ser entendido como camada de governança, acompanhamento e tramitação;
- o **assistente** continua sendo a camada de criação e estruturação do projeto.

4.2. Dois fluxos de projeto precisam coexistir

O modelo também precisa contemplar pelo menos dois cenários.

Cenário A — pesquisador já possui o próprio dataset

Nesse caso:

- ele quer estruturar o projeto;
- pode querer submeter para avaliação de conformidade e responsabilidade;

- não necessariamente precisa de sala segura;
- não necessariamente precisa do fluxo **data_clean**.

Cenário B — projeto depende de dados do NIAR

Nesse caso:

- a submissão pode envolver fluxo mais sensível;
- pode haver necessidade de engenharia de dados mais controlada;
- pode haver necessidade de sala segura em etapa posterior.

Consequência:

- nem todo projeto deve obrigatoriamente passar por **data_clean**;
- nem todo projeto deve obrigatoriamente envolver sala segura.

5. Problema de Modelagem Identificado

O principal problema atual é que:

- o assistente funciona bem para o próprio fluxo interno;
- mas o domínio administrativo da plataforma ainda não está explicitado de forma limpa.

Hoje:

- a sessão do assistente faz, ao mesmo tempo, papel de processo e de projeto;
- isso funciona tecnicamente para o assistente;
- mas dificulta a construção de uma camada administrativa mais clara.

Ao mesmo tempo:

- mexer demais no assistente agora pode atrasar ou complicar a evolução do admin.

Por isso, a solução precisa equilibrar:

- baixo impacto no assistente;
- melhoria de clareza no domínio;
- pouca redundância.

6. Direção de Modelo Proposta

6.1. Ideia principal

A proposta atual é introduzir uma entidade de **project**, mas de forma mínima.

A sessão do assistente continua existindo.

O projeto passa a ser:

- a entidade central de domínio para a plataforma;
- enquanto **wizard_session** continua sendo a entidade do fluxo guiado.

Assim:

- o projeto nasce do assistente;
- o admin passa a governar o projeto;
- a sessão continua sendo a origem operacional do conteúdo.

6.2. O que entra agora

projects

Entidade principal do domínio, mas inicialmente enxuta.

Campos mínimos sugeridos:

- **id**
- **owner_user_id**
- **source_wizard_session_id** ou vínculo equivalente
- **title**
- **created_at**
- **updated_at**

Objetivo:

- criar uma identidade estável para o projeto;
- sem exigir que o assistente já preencha todos os campos ricos do domínio.

wizard_sessions

Continua existindo, com a adição de:

- **project_id**

Isso permite:

- uma sessão pertencer a um projeto;
- um projeto possuir várias sessões;
- manter a lógica atual do assistente com mudanças menores.

project_status_history

Nova tabela administrativa para acompanhar o histórico do projeto.

Campos mínimos sugeridos:

- **id**
- **project_id**
- **status**
- **notes**
- **actor_user_id**
- **created_at**

Objetivo:

- registrar trilha de status sem misturar tudo dentro da sessão do assistente.

export_artifacts

Permanece existindo como hoje.

Ele continua sendo tratado como:

- estrutura técnica de artefatos gerados pelo assistente;
- e, no primeiro corte, pode inclusive servir como origem do documento anexado ao projeto.

6.3. O que não entra agora

Neste momento, não é necessário exigir já no **project** campos mais ricos como:

- **summary**
- **primary_objective**
- **secondary_objectives**

- **hypotheses**
- **study_type**
- **clinical_domain**
- **data_source_type**
- **requires_safe_room**

Motivo:

- esses campos podem até fazer sentido no domínio;
- mas para entrarem bem agora exigiriam mudança maior no assistente;
- e a prioridade atual é aproveitar o que já existe.

Esses atributos podem entrar depois, quando o assistente estiver pronto para preencher tudo isso de forma estruturada.

6.4. Sobre **data_source_type**

Há uma boa chance de, no futuro, esse atributo ser importante.

Mas hoje o assistente ainda trabalha isso de forma mais textual, especialmente na seção **data_sources**.

Portanto, a ideia mais saudável parece ser:

- primeiro criar o **project** mínimo;
- depois evoluir o assistente para extrair **data_source_type** de forma estruturada a partir do conteúdo do projeto.

7. Relações do Modelo Proposto

7.1. Relações principais

- **users 1:N projects**
- **users 1:N wizard_sessions**
- **projects 1:N wizard_sessions**
- **wizard_sessions 1:N export_artifacts**
- **projects 1:N project_status_history**

7.2. Interpretação dessas relações

projects 1:N wizard_sessions

Um projeto pode ter:

- uma sessão de documento do projeto;
- uma sessão de **data_clean**;
- outras sessões futuras, se necessário.

Cada sessão, por sua vez, pertence a um único projeto.

Essa é a razão pela qual:

- a solução mais simples é adicionar **project_id** diretamente em **wizard_sessions**;
- não há necessidade de tabela intermediária se cada sessão pertence a um único projeto.

8. Posição Atual sobre Documentos

8.1. Decisão de transição

No primeiro corte, não é obrigatório criar uma tabela formal de **project_documents**.

A decisão mais simples no momento é:

- manter **export_artifacts** como origem dos documentos gerados pelo assistente;
- usar especialmente o **artifact_type = project_docx** como documento principal inicial do projeto.

Isso reduz mudança no assistente.

8.2. Limitação dessa escolha

Hoje **export_artifacts** não guarda tudo.

Ele guarda principalmente arquivos gerados pelo sistema, como:

- **.docx** do projeto
- **.zip** de submissão
- **data_clean.py**

Mas ele não parece ser, hoje, o repositório formal de documentos enviados manualmente pelo usuário.

Então, se no futuro a plataforma precisar controlar:

- anexos enviados pelo pesquisador;
- versionamento de documentos;
- separação entre documento oficial e artefato técnico;

então provavelmente será necessário introduzir uma tabela mais específica de documentos.

9. Recomendações Práticas para a Próxima Etapa

9.1. O que fazer agora

Recomendação de implementação mínima:

1. criar **projects** de forma enxuta;
2. adicionar **project_id** em **wizard_sessions**;
3. manter **export_artifacts** como está;
4. criar **project_status_history**;
5. deixar enriquecimento de **project** para uma etapa posterior.

9.2. O que evitar agora

Evitar neste momento:

- reescrever profundamente o assistente;
- tentar estruturar todos os campos ricos do projeto já na primeira etapa;
- substituir **export_artifacts**;
- forçar um modelo administrativo completo antes de validar a convivência com o assistente.

9.3. Objetivo arquitetural

O objetivo deve ser:

- melhorar o domínio sem perder o que já funciona;

- permitir que o **admin** exista de forma mais organizada;
- preparar uma evolução futura em que o projeto tenha mais riqueza sem depender de uma refatoração abrupta.

10. Conclusão

O assistente atual já oferece uma base funcional importante.

Ele já resolve:

- autenticação;
- coleta guiada de conteúdo;
- geração de documento;
- geração de script de limpeza;
- exportação de artefatos;
- submissão em bundle.

Por isso, a estratégia mais saudável neste momento não é descartá-lo nem reestruturá-lo por completo.

A melhor direção é:

- introduzir um **project** mínimo;
- manter **wizard_session** como processo do assistente;
- adicionar camada de status e governança para o **admin**;
- evoluir gradualmente para um domínio mais rico conforme o assistente puder fornecer dados mais estruturados.

Essa abordagem reduz risco, preserva contexto e facilita o entendimento do sistema por participantes atuais do projeto e futuros mantenedores.